

Project Report 2: Little Chords Kindergarten Management Database

Database Systems CSCI 3423

Computer Science Division

Dr. Darwish

Alessandra FrancoLammoglia

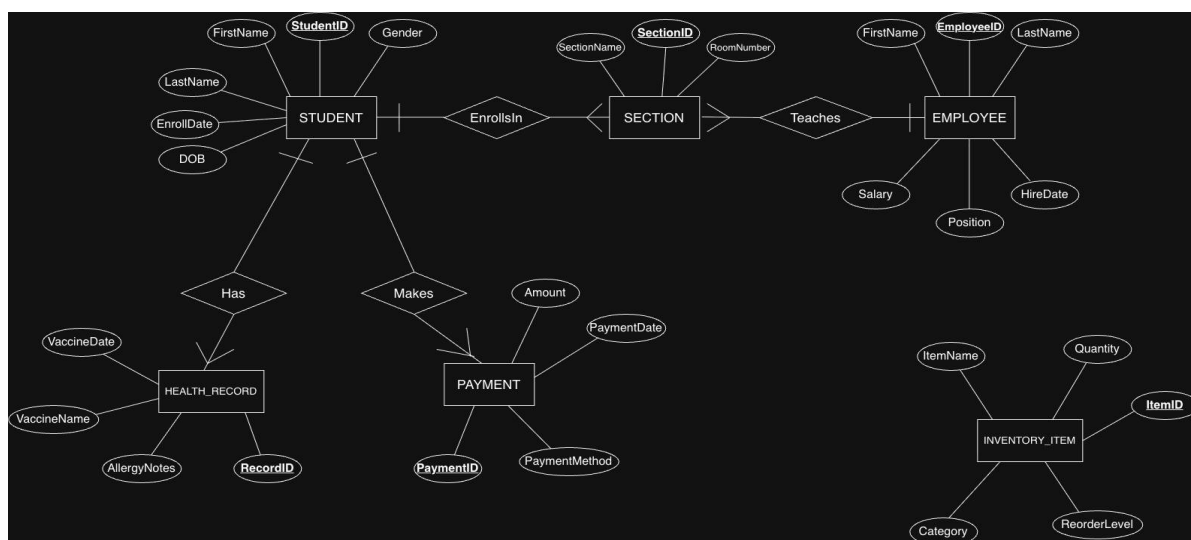
Database Name: Little Chords Kindergarten

Project Goals: The objective of the current project is to develop and create a relational database for Little Chords Kindergarten which will enable the management to manage the student enrollment process, the allocation of students to particular sections and teachers, to keep health records of the students, the tuition payments, and the classroom inventory.

Database Description: Little Chords Kindergarten database contains information on the students of the kindergarten, the sections they study in, the teaching personnel working in these sections, the health records of the students, the tuition payments and the inventory items used by the kindergarten. The described database is very useful to the administrative personnel since it makes the access to enrollment figures, vaccination records, payment histories and classroom inventories easier without using any paper records.

Data Model & Design

1. ER/EER Diagram:



2. Business Rules:

- Each student is registered in only one section but each section has one or more students.
- Each section is managed by only one employee but each employee can manage one or more sections.
- Each health record belongs to only one student but each student can have one or more health records.
- Each payment is paid by only one student but each student can make one or more payments.
- Inventory items are recorded separately and are not associated with any specific student, section, or employee.

3. Data Dictionary:

STUDENT

Column	Data Type	Restrictions	Description
<u>StudentID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each student
FirstName	VARCHAR(50)	NOT NULL	Student's first name
LastName	VARCHAR(50)	NOT NULL	Student's last name
DOB	DATE	NOT NULL	Student's date of birth
Gender	VARCHAR(10)	NOT NULL	Student's gender
EnrollDate	DATE	NOT NULL	Date the student enrolled
<i>SectionID</i>	INT	FOREIGN KEY (references SECTION)	Section the student belongs to

SECTION

Column	Data Type	Restrictions	Description
<u>SectionID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each section
SectionName	VARCHAR(30)	NOT NULL	Name of section
RoomNumber	VARCHAR(10)	NOT NULL	Room number assigned to the section
<i>EmployeeID</i>	INT	FOREIGN KEY (references EMPLOYEE)	Teacher assigned to the section

EMPLOYEE

Column	Data Type	Restrictions	Description
<u>EmployeeID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each employee
FirstName	VARCHAR(50)	NOT NULL	Employee's first name
LastName	VARCHAR(50)	NOT NULL	Employee's last name
Position	VARCHAR(30)	NOT NULL	Job title (teacher, assistant, admin)
HireDate	DATE	NOT NULL	Date the employee was hired
Salary	NUMERIC(10,2)	NOT NULL	Employee's annual salary

HEALTH_RECORD

Column	Data Type	Restrictions	Description
<u>RecordID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each health record
VaccineName	VARCHAR(50)	NOT NULL	Name of the vaccine administered
VaccineDate	DATE	NOT NULL	Date the vaccine was given
AllergyNotes	VARCHAR(255)	NULL	Notes on any known allergies
<i>StudentID</i>	INT	FOREIGN KEY (references STUDENT)	Student the record belongs to

PAYMENT

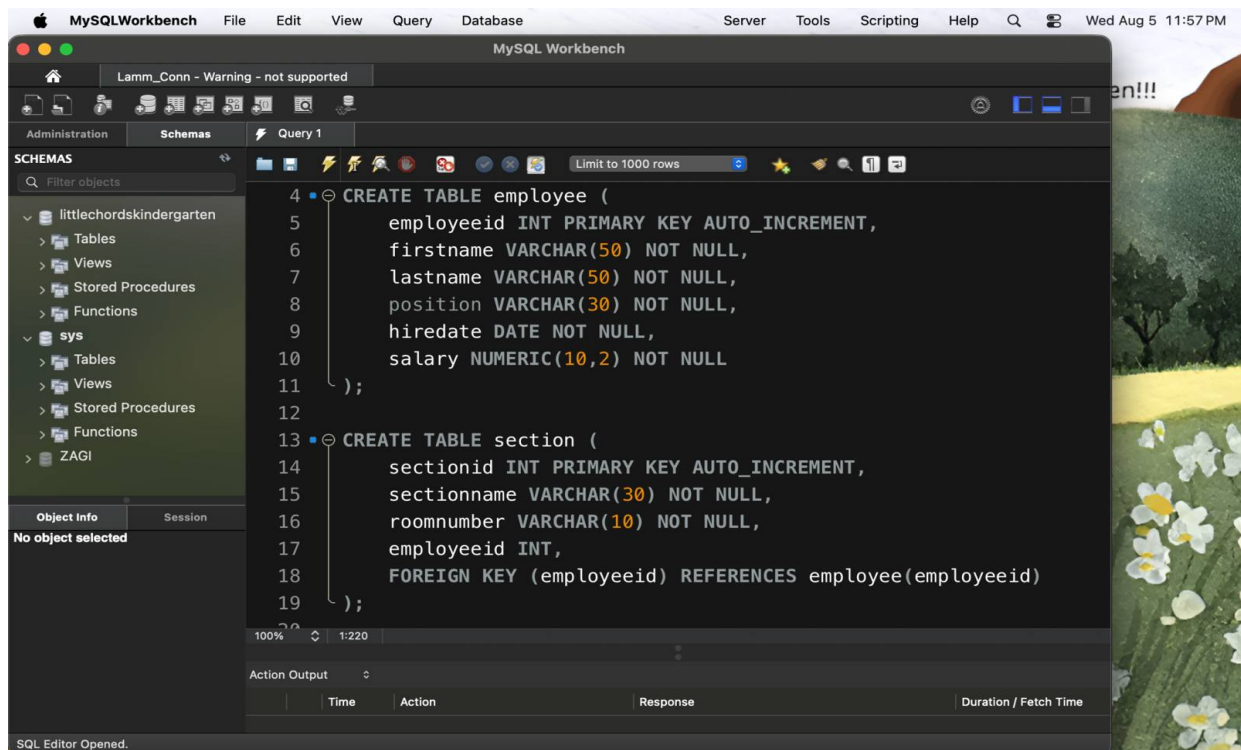
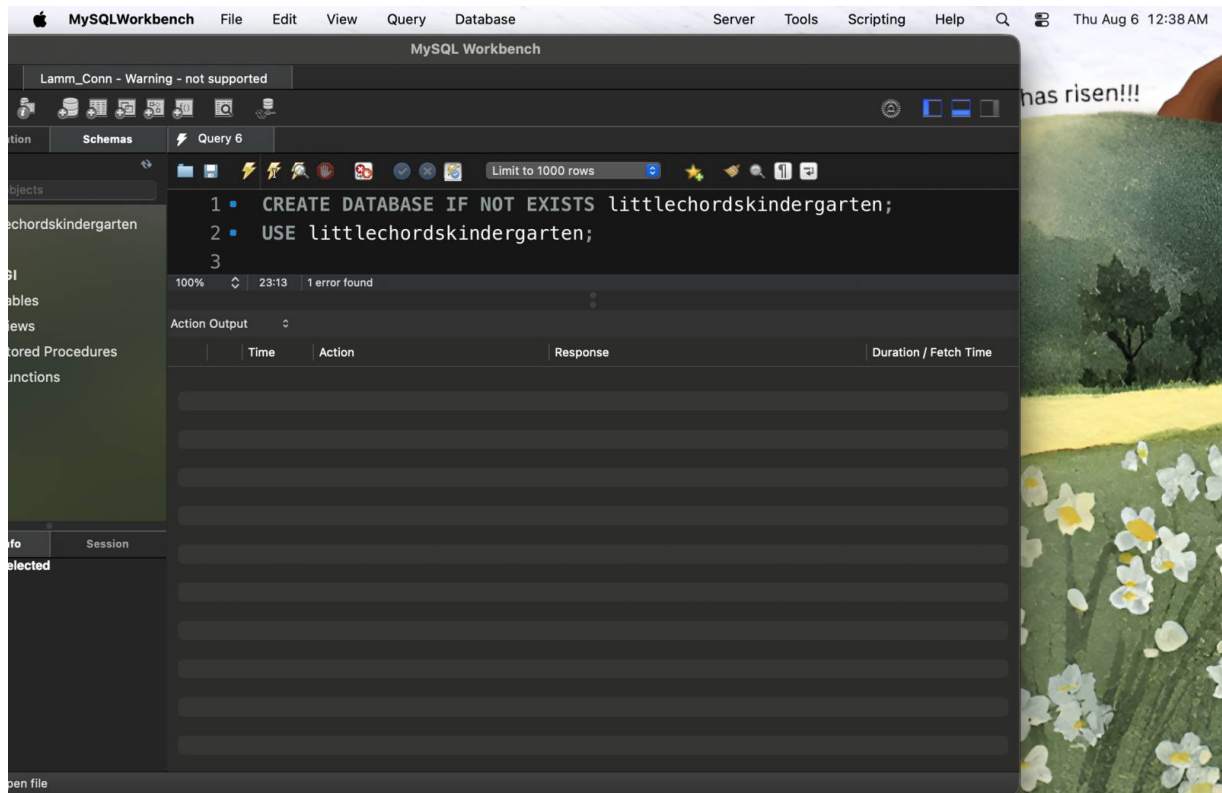
Column	Data Type	Restrictions	Description
<u>PaymentID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each payment
Amount	VARCHAR(8,2)	NOT NULL	Amount paid
PaymentDate	DATE	NOT NULL	Date payment was made
PaymentMethod	VARCHAR(20)	NOT NULL	Method used
<i>StudentID</i>	INT	FOREIGN KEY (references STUDENT)	Student the payment is for

INVENTORY_ITEM

Column	Data Type	Restrictions	Description
<u>ItemID</u>	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each item
ItemName	VARCHAR(50)	NOT NULL	Name of the item
Category	VARCHAR(30)	NOT NULL	Item category
Quantity	INT	NOT NULL	Quantity currently in stock
ReorderLevel	INT	NOT NULL	Threshold at which item should be reordered

Implementation

Database and Table Creation:



The screenshot shows the MySQL Workbench interface. The 'SCHEMAS' panel on the left lists databases: 'littlechordskindergarten', 'sys', and 'ZAGI'. The 'Query 1' editor contains the following SQL code:

```

21 CREATE TABLE student (
22     studentid INT PRIMARY KEY AUTO_INCREMENT,
23     firstname VARCHAR(50) NOT NULL,
24     lastname VARCHAR(50) NOT NULL,
25     dob DATE NOT NULL,
26     gender VARCHAR(10) NOT NULL,
27     enrolldate DATE NOT NULL,
28     sectionid INT,
29     FOREIGN KEY (sectionid) REFERENCES section(sectionid)
30 );
31
32 CREATE TABLE health_record (
33     recordid INT PRIMARY KEY AUTO_INCREMENT,
34     vaccinenname VARCHAR(50) NOT NULL,
35     vaccinedate DATE NOT NULL,
36     allergynotes VARCHAR(255) NULL,
37     studentid INT,
38     FOREIGN KEY (studentid) REFERENCES student(studentid)
39 );

```

The status bar at the bottom indicates 'SQL Editor Opened.' and '1:220'.

The screenshot shows the MySQL Workbench interface. The 'SCHEMAS' panel on the left lists databases: 'littlechordskindergarten', 'sys', and 'ZAGI'. The 'Query 1' editor contains the following SQL code:

```

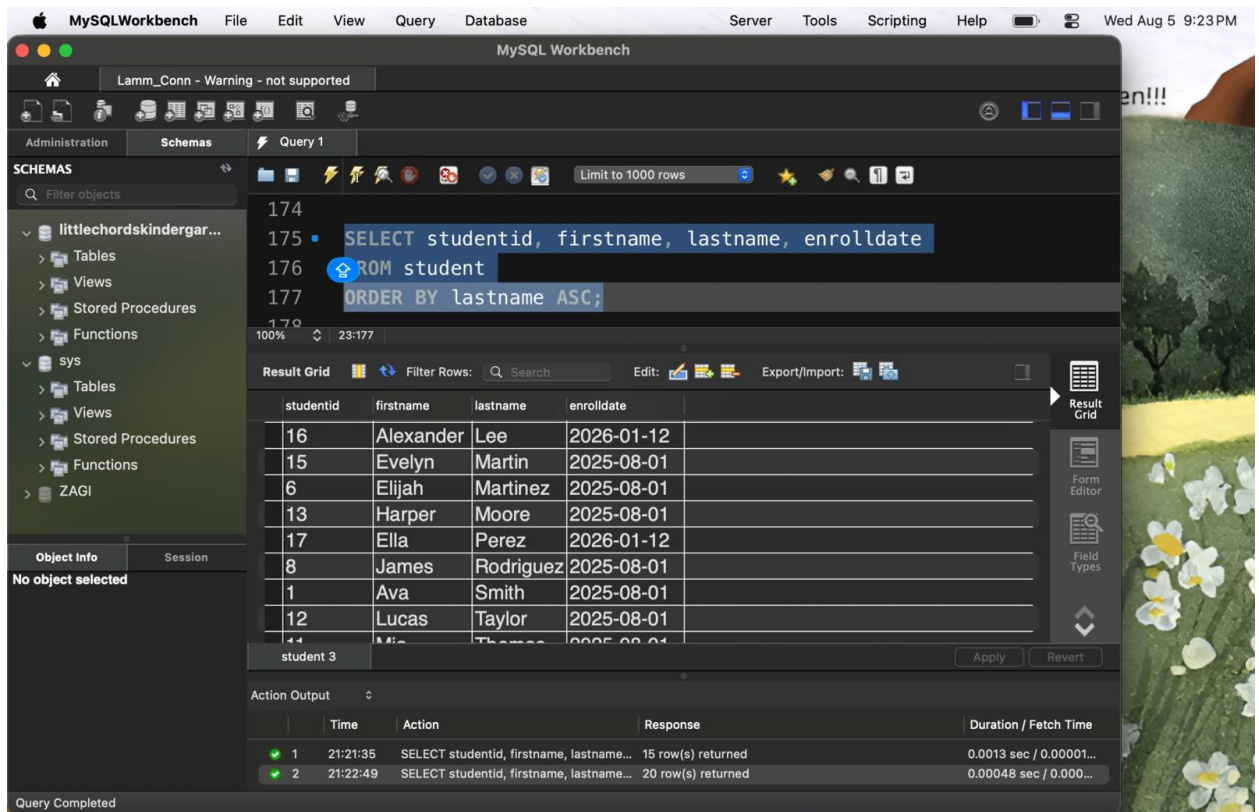
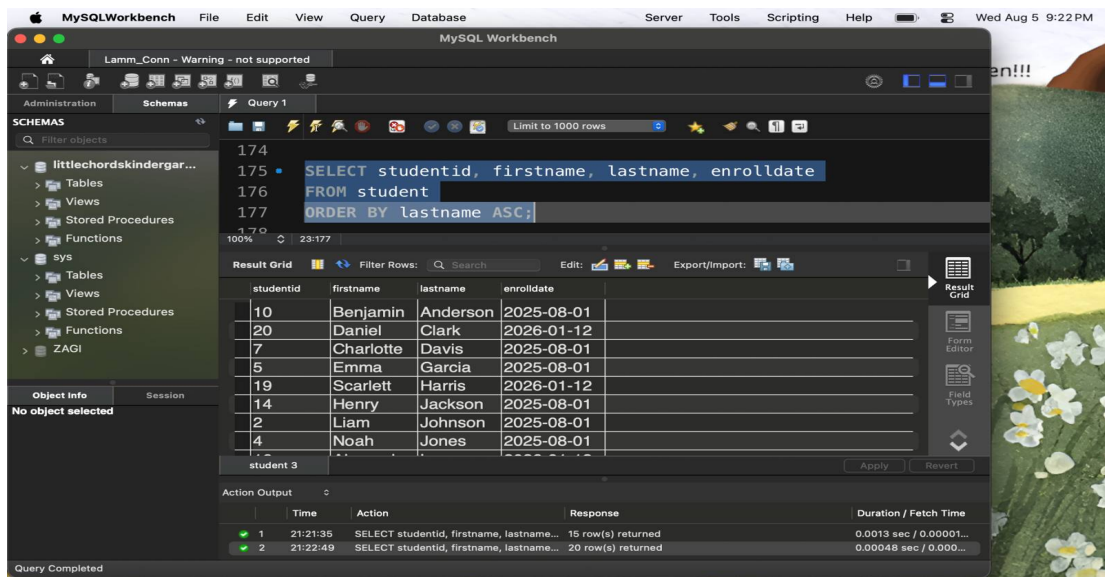
38
39 );
40
41 CREATE TABLE payment (
42     paymentid INT PRIMARY KEY AUTO_INCREMENT,
43     amount NUMERIC(8,2) NOT NULL,
44     paymentdate DATE NOT NULL,
45     paymentmethod VARCHAR(20) NOT NULL,
46     studentid INT,
47     FOREIGN KEY (studentid) REFERENCES student(studentid)
48 );
49
50 CREATE TABLE inventory_item (
51     itemid INT PRIMARY KEY AUTO_INCREMENT,
52     itemname VARCHAR(50) NOT NULL,
53     category VARCHAR(30) NOT NULL,
54     quantity INT NOT NULL,
55     reorderlevel INT NOT NULL
56 );
57

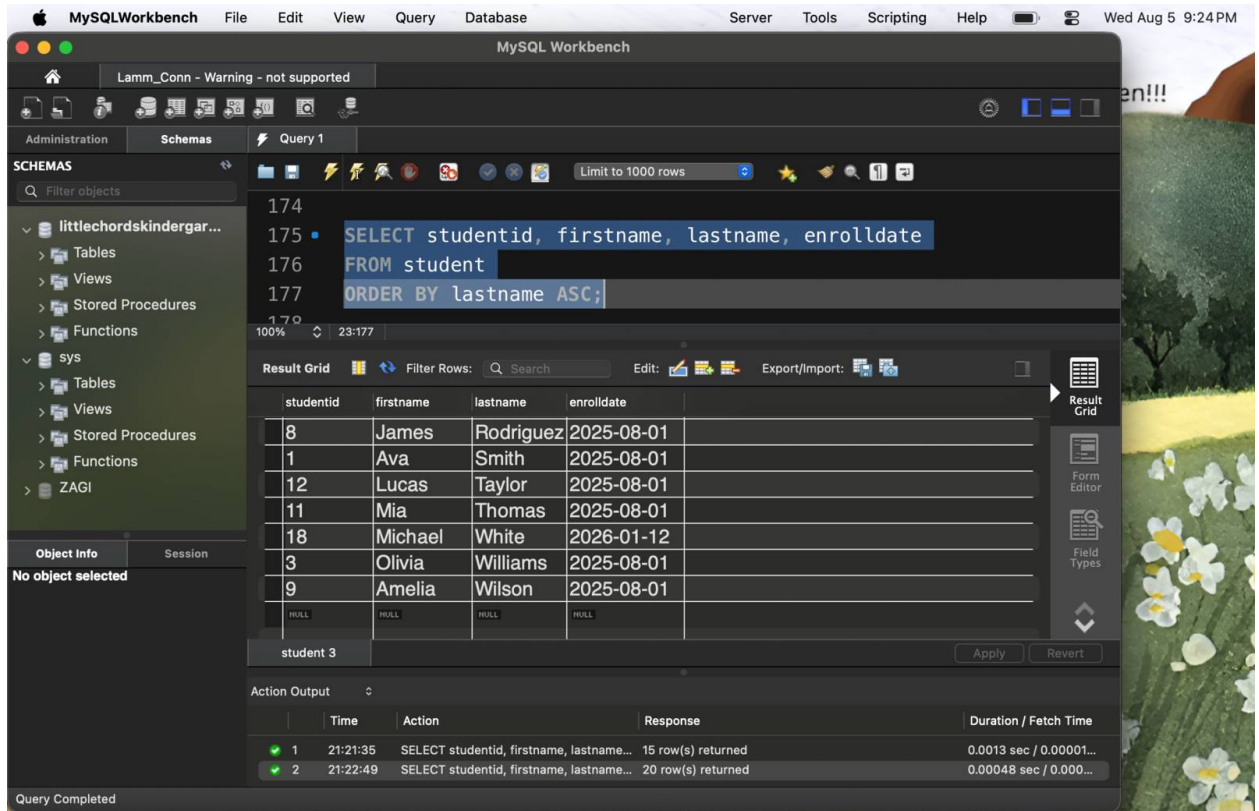
```

The status bar at the bottom indicates 'SQL Editor Opened.' and '1:220'.

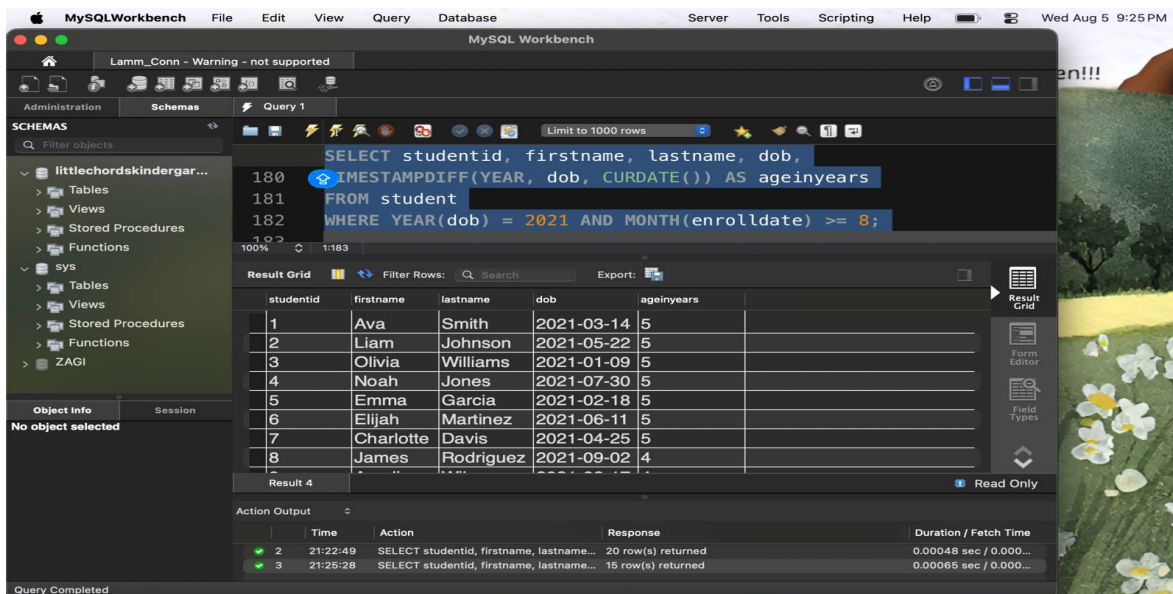
SQL Code: Include a snapshot of the SQL code used to create the database and tables in your report.

1. 1 simple select query with ordering. List of all students sorted alphabetically by surname.





2. 2 medium difficulty queries involving composite conditions, computations, aggregate functions, and grouping. Composite condition and computation. Students born in 2021, enrolled in section 1-5, in August or later with an age calculation.



The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
SELECT studentid, firstname, lastname, dob,
TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS ageinyears
FROM student
WHERE YEAR(dob) = 2021 AND MONTH(enrolldate) >= 8;
```

The result grid displays 15 rows of student data:

studentid	firstname	lastname	dob	ageinyears
8	James	Rodriguez	2021-09-02	4
9	Amelia	Wilson	2021-08-17	4
10	Benjamin	Anderson	2021-03-29	5
11	Mia	Thomas	2021-11-05	4
12	Lucas	Taylor	2021-01-27	5
13	Harper	Moore	2021-10-14	4
14	Henry	Jackson	2021-05-08	5
15	Evelyn	Martin	2021-12-19	4

The Action Output pane shows the execution of the query, indicating that 15 rows were returned.

Aggregation with grouping. Employees are grouped by position, counted and calculated the average salary.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

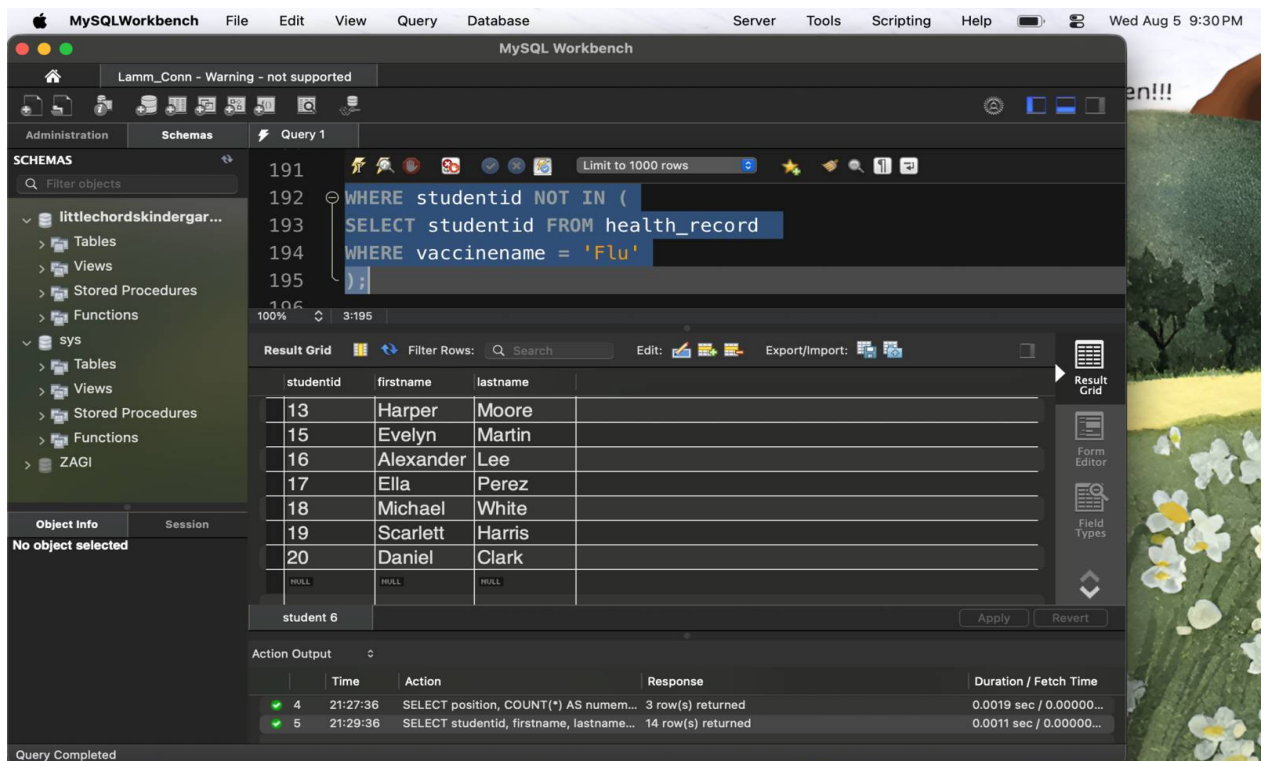
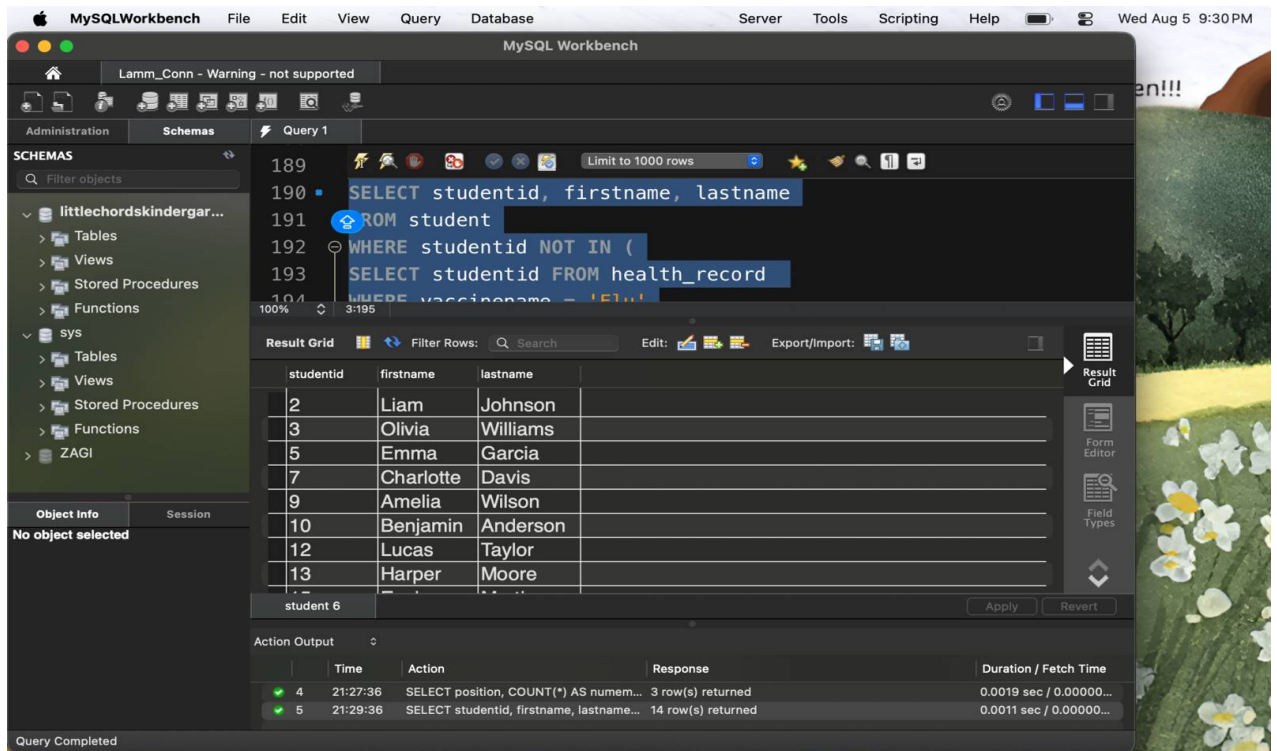
```
SELECT position, COUNT(*) AS numemployees,
AVG(salary) AS avgsalary
FROM employee
GROUP BY position;
```

The result grid displays 3 rows of aggregated data:

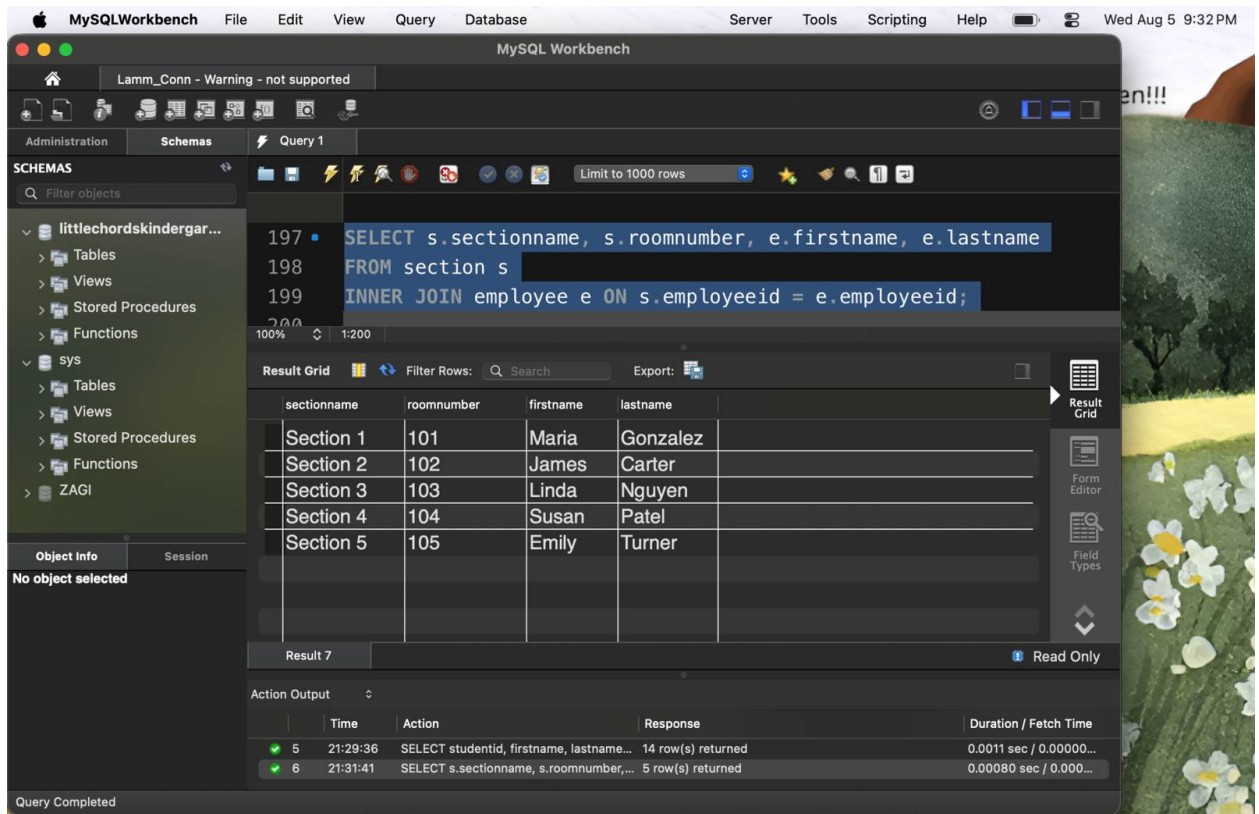
position	numemployees	avgsalary
Teacher	10	41650.000000
Assistant	7	31357.142857
Admin	3	47666.666667

The Action Output pane shows the execution of the query, indicating that 3 rows were returned.

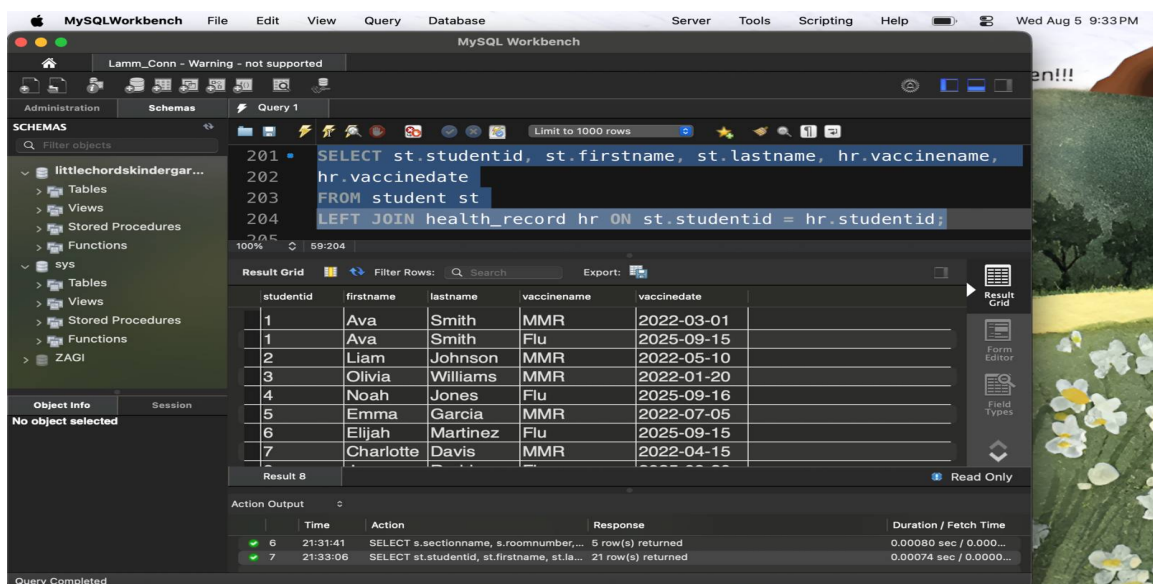
3. 1 subquery Students who have not received the Flu vaccine are identified based on the comparison of student ID and all students who have Flu vaccine recorded.



4. **join queries (1 inner join, 1 left or right outer join).** Inner join – Each class is matched with the employee who conducts this class.



Left outer join – list of all students with their health information, including students with no health records registered.



MySQL Workbench

Administration Schemas Query 1

SCHEMAS

Filter objects

littlechordskindergar...

Tables

Views

Stored Procedures

Functions

sys

Tables

Views

Stored Procedures

Functions

ZAGI

Object Info

Session

No object selected

201 SELECT st.studentid, st.firstname, st.lastname, hr.vacciname,

202 hr.vaccinedate

203 FROM student st

204 LEFT JOIN health_record hr ON st.studentid = hr.studentid;

100% 59:204

Result Grid

studentid	firstname	lastname	vacciname	vaccinedate
8	James	Rodriguez	Flu	2025-09-20
9	Amelia	Wilson	MMR	2022-09-01
10	Benjamin	Anderson	MMR	2022-03-25
11	Mia	Thomas	Flu	2025-09-18
12	Lucas	Taylor	MMR	2022-11-02
13	Harper	Moore	MMR	2022-10-10
14	Henry	Jackson	Flu	2025-09-15
15	Evelyn	Martin	MMR	2022-12-15

Result 8

Action Output

	Time	Action	Response	Duration / Fetch Time
6	21:31:41	SELECT s.sectionname, s.roomnumber,...	5 row(s) returned	0.00080 sec / 0.000...
7	21:33:06	SELECT st.studentid, st.firstname, st.la...	21 row(s) returned	0.00074 sec / 0.0000...

Query Completed

MySQL Workbench

Administration Schemas Query 1

SCHEMAS

Filter objects

littlechordskindergar...

Tables

Views

Stored Procedures

Functions

sys

Tables

Views

Stored Procedures

Functions

ZAGI

Object Info

Session

No object selected

201 SELECT st.studentid, st.firstname, st.lastname, hr.vacciname,

202 hr.vaccinedate

203 FROM student st

204 LEFT JOIN health_record hr ON st.studentid = hr.studentid;

100% 59:204

Result Grid

studentid	firstname	lastname	vacciname	vaccinedate
13	Harper	Moore	MMR	2022-10-10
14	Henry	Jackson	Flu	2025-09-15
15	Evelyn	Martin	MMR	2022-12-15
16	Alexan...	Lee	MMR	2022-02-02
17	Ella	Perez	MMR	2022-06-20
18	Michael	White	MMR	2022-04-06
19	Scarlett	Harris	MMR	2022-09-25
20	Daniel	Clark	HALL	HALL

Result 8

Action Output

	Time	Action	Response	Duration / Fetch Time
6	21:31:41	SELECT s.sectionname, s.roomnumber,...	5 row(s) returned	0.00080 sec / 0.000...
7	21:33:06	SELECT st.studentid, st.firstname, st.la...	21 row(s) returned	0.00074 sec / 0.0000...

Query Completed

5. 1 view (query must use a join, hint: you can reuse one of the previous queries). View of total payment of the tuition fee made by a student in addition to his/her section, sorted descending by the amount.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL query:

```
SELECT * FROM studentpaymentsummary ORDER BY totalpaid DESC;
```

The result grid displays the following data:

studentid	firstname	lastname	sectionname	totalpaid
1	Ava	Smith	Section 1	1000.00
2	Liam	Johnson	Section 1	1000.00
3	Olivia	Williams	Section 1	1000.00
4	Noah	Jones	Section 2	1000.00
5	Emma	Garcia	Section 2	1000.00
6	Elijah	Martinez	Section 2	500.00
7	Charlotte	Davis	Section 3	500.00
8	James	Rodriguez	Section 3	500.00

The Action Output pane shows the following log:

Time	Action	Response	Duration / Fetch Time
21:35:43	CREATE VIEW studentpaymentsummar...	Error Code: 1050. Table 'studentpaymentsummary' alr...	0.00060 sec
21:36:24	SELECT * FROM studentpaymentsum...	15 row(s) returned	0.00084 sec / 0.000...

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL query:

```
SELECT * FROM studentpaymentsummary ORDER BY totalpaid DESC;
```

The result grid displays the following data:

studentid	firstname	lastname	sectionname	totalpaid
8	James	Rodriguez	Section 3	500.00
9	Amelia	Wilson	Section 3	500.00
10	Benjamin	Anderson	Section 4	500.00
11	Mia	Thomas	Section 4	500.00
12	Lucas	Taylor	Section 4	500.00
13	Harper	Moore	Section 5	500.00
14	Henry	Jackson	Section 5	500.00
15	Evelyn	Martin	Section 5	500.00

The Action Output pane shows the following log:

Time	Action	Response	Duration / Fetch Time
21:35:43	CREATE VIEW studentpaymentsummar...	Error Code: 1050. Table 'studentpaymentsummary' alr...	0.00060 sec
21:36:24	SELECT * FROM studentpaymentsum...	15 row(s) returned	0.00084 sec / 0.000...

6. **1 union query** Combination of lists of staff employees hired before 2020 and those whose salaries exceed \$44,000. The purpose of their inclusion is listed.

The screenshot shows the MySQL Workbench interface with a query window displaying a UNION query. The query selects staff members hired before 2020-01-01 and those with salaries exceeding \$44,000. The result grid shows 10 rows of data.

```

216 SELECT firstname, lastname, 'Senior Staff' AS reason
217 FROM employee WHERE hiredate < '2020-01-01' UNION
218 SELECT firstname, lastname, 'High Earner' AS reason
219 FROM employee WHERE salary > 44000;

```

firstname	lastname	reason
Maria	Gonzalez	Senior Staff
Linda	Nguyen	Senior Staff
Susan	Patel	Senior Staff
Rachel	Lopez	Senior Staff
Emily	Turner	Senior Staff
Nancy	Foster	Senior Staff
Laura	Jenkins	Senior Staff
Patricia	Reed	Senior Staff

Result 10

Action Output

Time	Action	Response	Duration / Fetch Time
10 21:36:24	SELECT * FROM studentpaymentsumm...	15 row(s) returned	0.00084 sec / 0.000...
11 21:46:47	SELECT firstname, lastname, 'Senior St...	12 row(s) returned	0.0010 sec / 0.00001...

Query Completed

The screenshot shows the MySQL Workbench interface with a query window displaying a UNION query. The query selects staff members hired before 2020-01-01 and those with salaries exceeding \$44,000. The result grid shows 10 rows of data.

```

216 SELECT firstname, lastname, 'Senior Staff' AS reason
217 FROM employee WHERE hiredate < '2020-01-01' UNION
218 SELECT firstname, lastname, 'High Earner' AS reason
219 FROM employee WHERE salary > 44000;

```

firstname	lastname	reason
Emily	Turner	Senior Staff
Nancy	Foster	Senior Staff
Laura	Jenkins	Senior Staff
Patricia	Reed	Senior Staff
Susan	Patel	High Earner
Rachel	Lopez	High Earner
Thomas	Scott	High Earner
Patricia	Reed	High Earner

Result 10

Action Output

Time	Action	Response	Duration / Fetch Time
10 21:36:24	SELECT * FROM studentpaymentsumm...	15 row(s) returned	0.00084 sec / 0.000...
11 21:46:47	SELECT firstname, lastname, 'Senior St...	12 row(s) returned	0.0010 sec / 0.00001...

Query Completed

7. **1 custom stored function.** Function calculating age based on the date of birth of the student for each student in the list.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
1 SELECT studentid, firstname, lastname,
2    getstudentage(studentid) AS age
3 FROM student;
```

The result grid displays the following data:

studentid	firstname	lastname	age
1	Ava	Smith	5
2	Liam	Johnson	5
3	Olivia	Williams	5
4	Noah	Jones	5
5	Emma	Garcia	5
6	Elijah	Martinez	5
7	Charlotte	Davis	5
8	James	Rodriguez	4
9	Amelia	Wilson	4
10	Benjamin	Anderson	5

The Action Output pane shows the query completed successfully, returning 20 row(s) in 0.0012 seconds.

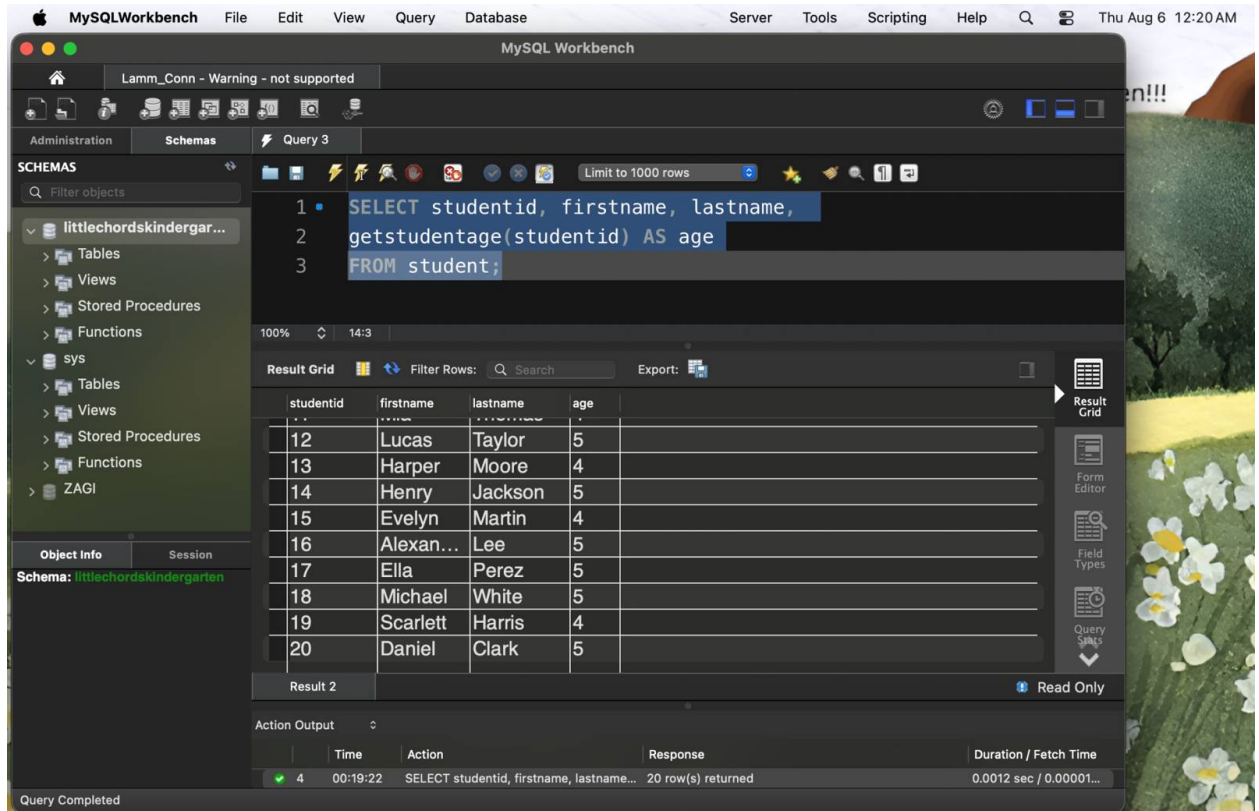
The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
1 SELECT studentid, firstname, lastname,
2    getstudentage(studentid) AS age
3 FROM student;
```

The result grid displays the following data:

studentid	firstname	lastname	age
10	Benjamin	Anderson	5
11	Mia	Thomas	4
12	Lucas	Taylor	5
13	Harper	Moore	4
14	Henry	Jackson	5
15	Evelyn	Martin	4
16	Alexan...	Lee	5
17	Ella	Perez	5
18	Michael	White	5
19	Scarlett	Harris	4

The Action Output pane shows the query completed successfully, returning 20 row(s) in 0.0012 seconds.



8. **1 custom stored procedure** Procedure displaying full list of students belonging to the specified section is defined and run for Section 1.

